

AMRITA
VISHWA VIDYAPEETHAM

SCHOOL OF ARTIFICIAL INTELLIGENCE

SEMESTER 03

23AID204 – ADVANCED DATA STRUCTURES AND ALGORITHMS

Title: Building a prefix trie based IP routing table for network packet forwarding

Project Report By

TEAM - 08

A Keerthana Chowdary [CB.AI.U4AID23001]

K KedharNadh [CB.AI.U4AID23015]

M Sai Krishna [CB.AI.U4AID23020]

M Rishitha [CB.AI.U4AID23023]

Introduction

In today's computer networks sending data packets quickly and correctly is really important and since sending packets in a big network is a very big challenge and we need to find a solution, Trie could very well be a good option and since it might usually take a lot of time to lookup so we store the routing table in the tries instead to reduce the lookup time and make the routing as efficient as possible. Routers use routing tables to decide where to send these packets. As networks get bigger, regular routing tables can become slow and hard to manage. A prefix tree or a trie can help organize IP addresses in a way that makes searching faster and easier because of how well it can perform the LPM. This report explores a simulated network of routers. Each router maintains a routing table and a Trie data structure for LPM. Dijkstra's computes the shortest path between routers ensuring optimal route selection. Each router maintains a Trie for efficient for LPM. Packets are forwarded hop-by-hop between routers using the routing table and Trie until the packet reaches its destination network or are dropped if no route exits.

Methodology

Algorithm

A. Network Topology Setup

We set up a network with five routers connected by links with specific costs and each router is assigned with a local network.

Router A: 192.168.1.0/24

Router B: 192.168.2.0/24

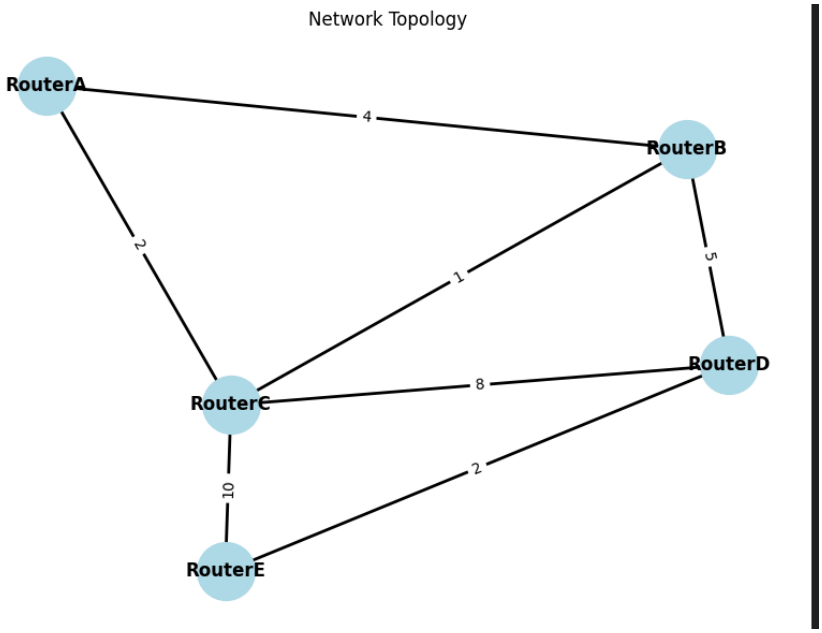
Router C: 192.168.3.0/24

Router D: 192.168.5.0/24

Router E: 192.168.0.0/16

To mimic actual costs in real world, the connections between these routers have different costs.

Figure 1: Network Topology



B. Building the Trie-Based Routing Table

Each router builds its own routing table using a trie after we calculate the shortest paths using the Dijkstra's algorithm.

Adding IP Prefixes:

- Each network destination is added to the trie from the routing table with the next hop information.
- The nodes in trie are navigated based on the bits of the IP prefix based on 0 or 1.

Trie Structure:

- Each node in the trie represents a bit (0 or 1) from the IP address after converting the IP address to octet(8 bit).
- If a node marks the end of a prefix, it stores the next hop info on where the packet has to be forwarded.

C. Computing Shortest Paths with Dijkstra's Algorithm

Dijkstra's algorithm is used to find the shortest paths between one router and all other routers:

- It calculates the shortest path from one router to all others.

- Takes into account of all the costs in each connection.
- Helps in filling up the next hop for each router for each destination.

D. Packet Forwarding Simulation

We simulate sending a packet, for example say, Router A to an IP address in Router E's network. Here iss how it goes:

- Starting Point: Packet starts at Router A.
- Next Hop: Router A uses the trie and looks up to find the best next hop.
- Forwarding: The packet moves hop by hop until it reaches the destination.

Implementation

A. Routing Table Generation:
   Each router computes routes to all other routers using dijstra's algorithm. The resulting routing table includes destination networks and their corresponding next hops.
B. Trie Data Structure in Detail

The trie is used to store and manage the routing table:

- Nodes (TrieNodes): Each node holds a bit from the IP prefix and has children for the next bits.
- Edges: Connect nodes based on the bit values of next child (0 or 1).
- Next Hop Information: Stored in nodes that mark the end of a prefix.

Why Tries are Better for IP Routing Tables:

- Fast Lookup Times: Searching takes time proportional to the number of bits in the IP address.
- Efficient Memory Usage: Shared prefixes reduce the amount of memory needed.
- Scalability: Handles many prefixes without slowing down.

Dynamic Updates: Easy to add or remove routes as the network changes

Time Complexity Analysis

A. Trie Operations

1. Insertion

- Operation: Adding an IP prefix to the trie.
- Analysis: Each bit of the prefix is checked once.
- Time Complexity: O(n), where n is the prefix length in bits (length of subnet). since it just has to traverse through each node and check for a node with no child and add it there at the end and n is the number of bits in the prefix (32 for IPv4).

2. Search (Longest Prefix Match)

- Operation: Finding the best matching prefix in the trie.
- Analysis: Each bit of the IP address is checked until no match is found.
- Time Complexity: O(m), where m is the IP address length in bits (32 for IPv4). Same as the insertion, its basically traversing through the trie and comparing the bits(LPM)

B. Dijkstras Algorithm

- Operation: Finding the shortest paths from a starting router to all others.
- Analysis: Uses a heap to efficiently get the next closest router.
- Time Complexity: O((V+E)log V), where E is the number of connections and V is the number of routers. The minimum distance node and edges for all neighbors takes O(log V) per node in single execution. For V and E the time complexity is O((V+E) log V).

C. Overall Routing Table Construction

- Trie Construction: For each router, adding all needed prefixes takes O(k * n) time, where k is the number of prefixes and n is the prefix length.
- Shortest Path Calculation: Running Dijkstra's for each router takes O(V *(V+ E) log V) time in total. The algorithm is run once for each router to compute the routing table where V is the number of routers and E is number of connections.

Observation:

The implementation simulates a network topology of interconnected routers connected to its neighbors with specific costs (Weights). By using Dijkstra's Algorithm for routing table and a Trie data structure for Longest Prefix Match (LPM) in packet forwarding. The routing table reflects the shortest paths based on the link costs and Trie efficiently handle IP prefix-based lookups. Packet forwarding simulations confirm that packets follow the shortest path and correctly identify the unreachable destinations or routing loops.

Conclusion

Using a trie-based IP routing table makes routing faster and more efficient. Tries help quickly find the best route for packets, and Dijkstra's algorithm ensures the routes are the shortest possible. Compared to other methods, tries are better at handling lots of routes, use memory wisely, and are easy to update. This shows that tries are a great choice for managing IP routes in big networks.

References

1]J. Smith, "A Fast Trie-based IP Lookup Algorithm," M.S. thesis, Dept. of Electronics Engineering, New Mexico State University-Main Campus, Las Cruces, NM, USA.

2] T. Kijkanjanarat and H. J. Chao, "Fast IP routing lookups for high performance routers".